

PyNotebook

Software Requirements Specification

Version 1.0

February 20, 2026

Chris O'Brien

Team Leader and Designer

Andrew Presta

Programmer and Designer

Vincent Tran

Programmer and Designer

PyNotebook

Revision History

Date	Description	Author	Comments
1/28/26	Version 0.1	Chris, Andrew, Vince	Document Creation
2/20/26	Version 1.0	Team Members	First Submission

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date

PyNotebook

Table of Contents

1. INTRODUCTION.....	1
1.1 PURPOSE	1
1.2 SCOPE.....	1
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS.....	1
1.4 OVERVIEW.....	1
2. GENERAL DESCRIPTION	2
2.1 PRODUCT PERSPECTIVE.....	2
2.2 PRODUCT FUNCTIONS.....	2
2.3 USERS AND CHARACTERISTICS	2
2.4 GENERAL CONSTRAINTS	2
2.5 ASSUMPTIONS AND DEPENDENCIES	2
2.6 OPERATING ENVIRONMENT	3
3. SPECIFIC REQUIREMENTS	3
3.1 EXTERNAL INTERFACE REQUIREMENTS.....	3
3.1.1 User Interfaces	3
3.1.2 Hardware Interfaces	3
3.1.3 Software Interfaces	3
3.1.4 Communications Interfaces	3
3.2 FUNCTIONAL REQUIREMENTS	3
3.2.1 Lessons or Guidelines for coding students	3
3.2.2 In-App IDE.....	3
3.2.3 Quiz.....	4
3.2.4 General Interfacing within the Main Lesson.....	4
3.2.5 View Generated Test Questions.....	4
3.2.6 Multiple Choice Selection	5
3.2.7 Test Submission.....	5
3.2.8 Forced Manual Code Input.....	5
3.2.9 Category.....	6
3.2.10 Repeat Lesson.....	6
3.3 USE CASES	6
3.3.1 Use Case #1	6
3.3.2 Use Case #2.....	6
3.3.3 Use Case #3.....	7
3.3.4 Use Case #4.....	7
3.3.5 Use Case #5.....	7
3.3.6 Use Case #6.....	8
3.3.7 Use Case #7.....	8
3.3.8 Use Case #8.....	9
3.4 NON-FUNCTIONAL REQUIREMENTS.....	9
3.4.1 Performance	9
3.4.2 Reliability	9
3.4.3 Availability	9
3.4.4 Security.....	9
3.4.5 Maintainability.....	10
3.4.6 Portability.....	10
3.5 DESIGN CONSTRAINTS.....	10
4. ANALYSIS MODELS	10
4.1 SEQUENCE DIAGRAMS	10
4.1.1 Diagram for Use Case #1.....	10
4.1.2 Diagram for Use Case #2.....	11
4.1.3 Diagram for Use Case #3.....	12
4.1.4 Diagram for Use Case #4.....	12
4.1.5 Diagram for Use Case #5.....	13
4.1.6 Diagram for Use Case #6.....	13

PyNotebook	
4.1.7 <i>Diagram for Use Case #7</i>	14
4.1.8 <i>Diagram for Use Case #8</i>	14
5. CHANGE MANAGEMENT PROCESS.....	15
REFERENCES.....	15
A. APPENDICES	15
A.1 APPENDIX 1.....	15
A.2 APPENDIX 2 – MEETING NOTES.....	15

1. Introduction

The current world of beginner and intermediate coding students is shrouded in many different learning options online. These options tend to promote a lack of deep, low-level knowledge. Instead, vying for fill-in-the-blank block coding and copy and paste tutorials. We believe that, to unlock a deeper understanding of your code, students must manually type their solutions and be assessed on their knowledge of what they wrote. This problem is currently more integral than ever with the rise of AI code solutions and “vibe-coding”. Our solution is a manual learning environment that forces students to code their own solution and have them assessed their code to promote in-depth learning.

1.1 Purpose

This SRS pertains to any engineers, designers, and outbound clientele connected to the PyNotebook product. This document is a self-encompassing guide of all features, requirements, and use cases for which to design and implement this software. All measures agreed upon are to be built at high priority for the 1.0 release of the product unless edited within this document (see revision history for changes after Version 1.0).

1.2 Scope

The software product, PyNotebook is a desktop education tool designed to support students learning introductory programming concepts. The software will accept Python code that students submit, analyze it to identify programming concepts, generate a multiple-choice quiz based on concept categories, and provide any additional education features. The software will not function as a full IDE or support programming languages other than Python. The primary goal of the software is to improve student learning through concept identification and assessment creation. This can limit the responsibilities of professors and make it easier for students to learn new concepts.

1.3 Definitions, Acronyms, and Abbreviations

<This subsection should provide the definitions of all terms, acronyms, and abbreviations required to properly interpret the SRS. This information may be provided by reference to one or more appendixes in the SRS or by reference to other documents.>

IDE – Integrated Development Environment, a basis for typing in code (so that it can be compiled, or ran)

GitHub – Allows developers to create, store, manage, and share their code

Python – Coding language in which will be taught (the software itself will also be coded in Python)

Compile – a way of processing the code

1.4 Overview

The remainder of this document is broken into three additional chapters and an appendix. Chapter 2 focuses on the general function of the software, in addition to its constraints. The chapter also provides information about the expected userbase for the software and some general assumptions/prerequisites for the implementation.

Chapter 3 focuses on highlighting the specific requirements and functions of the software brought on by the client. This is a more detailed look into the general functionality of the software, as well as various use cases in which we see the software fits its description.

Chapter 4 highlights various analysis models that will help determine the functionality of the software. The models should be read in conjunction with Chapter 3 to have a full understanding of what PyNotebook has to offer.

Finally, the appendices withhold additional information regarding the document. **(This should be added onto further after writing the appendices)**

2. General Description

2.1 Product Perspective

PyNotebook is a fully stand-alone, original product. In which, there are no additional products and programs being used in its implementation

In addition, PyNotebook will be a desktop application to be downloaded from our GitHub site. Due to the nature of holding various lessons for its users, a database is necessary for storing progress for multiple users across different sessions.

2.2 Product Functions

The general functions of PyNotebook are bulleted below. Please note that specific requirements are branching off the general topics listed here.

- PyNotebook is a desktop application, with a built-in IDE for code input
- There will be various lessons about simple topics pertaining to Python and programming
- During the lesson, a user is not allowed to paste in code and must type it manually
- After submitting their code and having the correct output, the user will be asked more in-depth questions about the main subject
- After the completion of a quiz, the user is allowed to move onto the next subject matter

2.3 Users and Characteristics

Students (favored)

- Users that are currently learning programming (particularly Python) that range from beginner to intermediate level coders
- Enrolled in introductory programming courses

Teachers

- Use the product as a supplemental educational tool to assess student understanding and reinforce learning outcomes
- Teach introductory programming courses

Self-Taught Programmers

- Use the product outside of an academic setting to deepen their understanding of programming concepts and to cut out over-reliance on copy-and-pasted solutions
- Not enrolled in the courses that the students and teachers are in

2.4 General Constraints

CO-01: Only one coding language will be offered

CO-02: Core learning features will be prioritized over advanced explanations/personal feedback

CO-03: The development of the project will cease to continue at the end of the academic year

2.5 Assumptions and Dependencies

AS-01: The system is used by beginner-level programmers, able to copy, paste, and write basic code.

AS-02: Users have access to a desktop that can run the executable

DE-01: Availability of a Python runtime capable of compiling and validating submitted code

DE-02: Access to required Python parsing and analysis libraries

DE-03: Availability for GitHub executable distribution

2.6 Operating Environment

The software will operate on a desktop computer capable of running a Python executable file. The system is intended to run on modern operating systems including Window, macOS, and Linux. The application does not require any external software systems beyond required Python libraries. Additionally, no special hardware is required.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

3.1.2 Hardware Interfaces

3.1.3 Software Interfaces

3.1.4 Communications Interfaces

3.2 Functional Requirements

This section describes specific features of the software project. If desired, some requirements may be specified in the use-case format and listed in the Use Cases Section.

3.2.1 Lessons or Guidelines for coding students

3.2.1.1 Description and Priority

The software has a basic guide to learning general Python coding. These lessons are ordered by importance, and each section gives a brief description of various topics within the subject. The goal is for the student to write a piece of code that contains the features that they have read about.

Priority: Medium

3.2.1.2 Stimulus/Response Sequences

- User opens the software to learn coding
- Software has a general guide to learning as a beginner
- User selects a lesson to start
- Software opens the main window which shows a lesson and IDE

3.2.1.3 Functional Requirements

This requirement seems to be a standalone feature that is not crucial to the workings of the other features of the software

3.2.2 In-App IDE

3.2.2.1 Description and Priority

Within the application, the user must be able to input their code into an IDE which will compile the code given and report errors back to the user.

Priority: High

3.2.2.2 Stimulus/Response Sequences

- User has a solution to a coding lesson

- User writes solution into the IDE
- Software compiles the code
- Code is checked for error and result is passed as output to the user

3.2.2.3 Functional Requirements

3.2.3 Quiz

3.2.3.1 Description and Priority

This feature has the user take quizzes to test their coding knowledge on a certain subject after learning it.

Priority: High

3.2.3.2 Stimulus/Response Sequences

- User must submit the answers to the quiz
- User has a solution to all quiz question

3.2.3.3 Functional Requirements

FR-03.1: The Multiple-Choice Selection should be available and working for the Quiz to work

FR-03.2: The Test Submission should be available and working for the Quiz to work

3.2.4 General Interfacing within the Main Lesson

3.2.1.1 Description and Priority

The interfacing once a user has selected a lesson should contain, the lesson within a box, the IDE for input, and eventually a section for quiz questions after code has been inputted.

Priority: Medium

3.2.1.2 Stimulus/Response Sequences

- User selects lessons to get into the main window
- System opens the main interfaced window that shows the lesson and IDE
- User completes the lesson with code input
- System changes the interfacing to show the quiz questions

3.2.1.3 Functional Requirements

FR-4.1: The IDE should be available and working for the interfacing to work

FR-4.2: The user code input should be able to be scanned and generate quiz questions in order for the interfacing to show them

3.2.5 View Generated Test Questions

3.2.5.1 Description and Priority

This feature allows the test taker to view all generated test questions on a single page.

Priority: High

3.2.5.2 Stimulus/Response Sequences

- System retrieves the generated questions

- System displays all questions on a single page

3.2.5.3 Functional Requirements

FR-5.1: The test taker must have submitted their code.

3.2.6 Multiple Choice Selection

3.2.6.1 Description and Priority

This feature allows the test taker to select an answer from four multiple choice options for each question.

Priority: High

3.2.6.2 Stimulus/Response Sequences

- System retrieves the generated answers
- System displays four answers per question
- Test taker selects an option

3.2.6.3 Functional Requirements

FR-6.1: The test taker must be able to view generated test questions.

3.2.7 Test Submission

3.2.7.1 Description and Priority

This feature allows the test taker to submit a completed test and receive feedback on their performance. Feedback includes the number of correctly answered questions.

Priority: High

3.2.7.2 Stimulus/Response Sequences

- Test taker submits a test with all questions answered
- System evaluates the selected answers
- System displays feedback with the number of correct answers

3.2.7.3 Functional Requirements

FR-7.1: The test taker must have an answer selected for all questions..

3.2.8 Forced Manual Code Input

3.2.8.1 Description and Priority

The user will be forced by the software to manually input their code, restricting them from pasting code from generators or other sources.

3.2.8.2 Stimulus/Response Sequences

- User find a solution to the coding lessons
- User attempts to paste in solution to the IDE
- System flags the solution for a paste function
- System forced user to retype their solution manually

3.2.8.3 Functional Requirements

FR-8.1: If the IDE cannot take code input, there is no reason to restrict the code input from the user

3.2.9 Category

3.2.3.1 *Description and Priority*

Within the application, the user's code will be parsed into different categories.

Priority: Medium

3.2.3.2 *Stimulus/Response Sequences*

- User must submit code

3.2.3.3 *Functional Requirements*

FR-09.1: The user's code must be able to be parsed.

3.2.10 Repeat Lesson

3.2.3.1 *Description and Priority*

This feature allows the user to retake a quiz for a lesson they have already previously completed.

Priority: Low

3.2.3.2 *Stimulus/Response Sequences*

- User must submit the answers to the quiz
- User must go back to a quiz for a lesson they have previously completed

3.2.3.3 *Functional Requirements*

FR-10.1: A previous lesson must already be completed to use this function

...

3.3 Use Cases

3.3.1 Use Case #1

Use Case Name	View Test Questions
Reference	3.2.5
Trigger	The test taker clicks a submit button after inputting Python code.
Precondition	The test taker has submitted code and the system successfully generated test questions.
Basic Path	1. The system navigates the test taker to a new test page. 2. The system retrieves the generated test questions. 3. The system displays all test questions on a single page.
Alternative Paths	In step 2, if no questions were generated, the system displays a message indicating that no test is available.
Postcondition	All generated test questions are visible to the test taker on a new page.
Exception Paths	The test taker may exit the test page at any time.
Other	

Table 1: Use case #1

3.3.2 Use Case #2

Use Case Name	View Categories of A Quiz
Reference	3.2.3, 3.2.9
Trigger	The test taker clicks a submit button after inputting Python code.
Precondition	The test taker must had submitted code.

Basic Path	1. The test taker submits their code 2. The system parses the code into different categories
Alternative Paths	In step 2, if the code is not able to be parsed, an error message will be made.
Postcondition	The code is parsed into separate categories and is visible to the user.
Exception Paths	
Other	

*Table 2: Use case #2***3.3.3 Use Case #3**

Use Case Name	Retake Quiz
Reference	3.2.3, 3.2.10
Trigger	The test taker clicks a button to retake a quiz.
Precondition	The test taker must have already finished a prior quiz.
Basic Path	1. The student wishes to redo a quiz. 2. The student navigates to the previous lesson they want to redo. 3. The system allows the user to redo a prior quiz.
Alternative Paths	In step 2, if the user had not already completed a prior quiz, then an error message will occur.
Postcondition	The user will be allowed to retake a quiz.
Exception Paths	
Other	

*Table 3: Use case #3***3.3.4 Use Case #4**

Use Case Name	Using IDE
Reference	3.2.1, 3.2.2, 3.2.4
Trigger	Selected a Lesson
Precondition	This seems to be the baseline line function for the while program
Basic Path	1. Student wishes to complete a lesson's problem 2. Built-In IDE is displayed for their code input 3. Student inputs their code manually (See 3.3.5 Use Case #6) 4. Student confirms their submission using the submit button 5. System will process code through the IDE and scanning function 6. System's IDE will show any errors or correct output if everything is clean
Alternative Paths	In Step 5, the IDE may find errors within the code. If so, the error will be displayed as per usual IDE practice. The user will fix the errors for another submission
Postcondition	The scanner function of code is ready and working
Exception Paths	
Other	

*Table 4: Use case #4***3.3.5 Use Case #5**

Use Case Name	Manual Code Input
----------------------	-------------------

Reference	3.2.8
Trigger	Selected a Lesson
Precondition	Built-In IDE is ready and working
Basic Path	1. Student begins a solution to a lesson problem within the IDE 2. Student pastes a solution they found online 3. System recognizes the code as pasted, due to the speed at which it was entered 4. System will flag the code as invalid and force the user to retype their solution
Alternative Paths	In Step 2, the student may use other forms of code examples to paste into their solution (maybe examples built into the text-box lesson). The system will still flag all those inputs as invalid and run the normal procedure as it would.
Postcondition	The scanner function of code is ready and working
Exception Paths	
Other	

*Table 5: Use case #5***3.3.6 Use Case #6**

Use Case Name	Lesson Interfacing
Reference	3.2.1, 3.2.4
Trigger	Selected a Lesson
Precondition	This seems to be one of the baseline line functions for the whole program
Basic Path	1. Student selects a lesson from a main window 2. System displays the interfaced lesson window, which shows the lesson, IDE 3. Student may decide to complete the lesson's problem, or read the guidelines from the textbox
Alternative Paths	As stated, Step 3 leads into different paths. The student may decide to just read the lesson, or they can complete the code solution. Paths involving the code are discussed in other use cases (See 3.3.4 for more)
Postcondition	
Exception Paths	
Other	

*Table 6: Use case #6***3.3.7 Use Case #7**

Use Case Name	Select Answer
Reference	3.2.6
Trigger	The test taker views a generated test.
Precondition	Generated test questions are displayed to the test taker.
Basic Path	1. The system displays a test question with four answer options. 2. The test taker selects one answer option for the question. 3. The system records the selected answer. 4. Steps 1-3 repeat until all questions have a selected answer.
Alternative Paths	The test taker changes a previously selected answer.
Postcondition	Each test question has one selected answer.
Exception Paths	The test taker may leave the test before answering all questions.
Other	

Table 7: Use case #7

3.3.8 Use Case #8

Use Case Name	Submit Test
Reference	3.2.7
Trigger	The test taker selects the submit test option.
Precondition	All test questions have a selected answer.
Basic Path	1. The test taker submits the completed test. 2. The system evaluates the selected answers. 3. The system calculates the number of correct answers. 4. The system displays feedback for the number of correct answers.
Alternative Paths	If the questions weren't all answered, the system prevents submission and displays an error message.
Postcondition	The test taker receives feedback indicating the number of correctly answered questions.
Exception Paths	The test taker exits the application before submission is completed.
Other	

Table 1: Use case #8

3.4 Non-Functional Requirements

3.4.1 Performance

Our main goal for performance will be in maintaining the speed of a normal compile within an IDE. In addition to the compile, we would like to generate our test questions in an adequate time. We do not think we can generate questions during compilation time, but we are looking for a delay as small as possible so that the code output can be generated alongside the questions being posted.

3.4.2 Reliability

We believe that our software will have limited issues on the side of reliability. Many factors including pages amount, desktop usage, and IDE implementation will create a reliable program. Our goal is that the user experiences a crash once in a 30-day period, with issues arising from large code input being given to our scanners. With a crash, we expect some memory loss depending on our saving procedures.

3.4.3 Availability

We believe that our software will have limited issues on the side of availability. Factors, including internet access, web browser access, and access to a computer will create an available program. Our goal is that all potential users of the program will be able to readily access it via a web browser. The program should always be available and functioning properly.

3.4.4 Security

Our main goal for security will be to prevent unauthorized write/delete access from anyone who is not the owner of the user account. There will be measures in place to ensure that the denial to "copy and paste" feature cannot be bypassed. There is no restriction on read access as long as that lesson is unlocked. The device on which the PyNotebook program will be run on will have to have its own security. Only the user will have physical access to the device and the program on it. There is no other special protection built into the system, except for giving the user write access to the PyNotebook program for them to write code.

3.4.5 Maintainability

In order to preserve code quality and reusability, we will organize code into clear components and simple naming conventions. Documentation and code comments will be provided to ensure future developers understand the system. Changes to one system should not affect another. Components should be designed in a way that is scalable. We shall follow optimal coding practices and version control procedures to support long-term maintenance.

3.4.6 Portability

Our system will be designed to operate on multiple operating systems, including macOS and Windows. The application will not include libraries that are dependent on a specific operating system. All dependencies will be documented in case libraries change system requirements. The program does not require special hardware and will run using standard Python installations. Transitioning the application between supported operating systems will require minimal modification to the code.

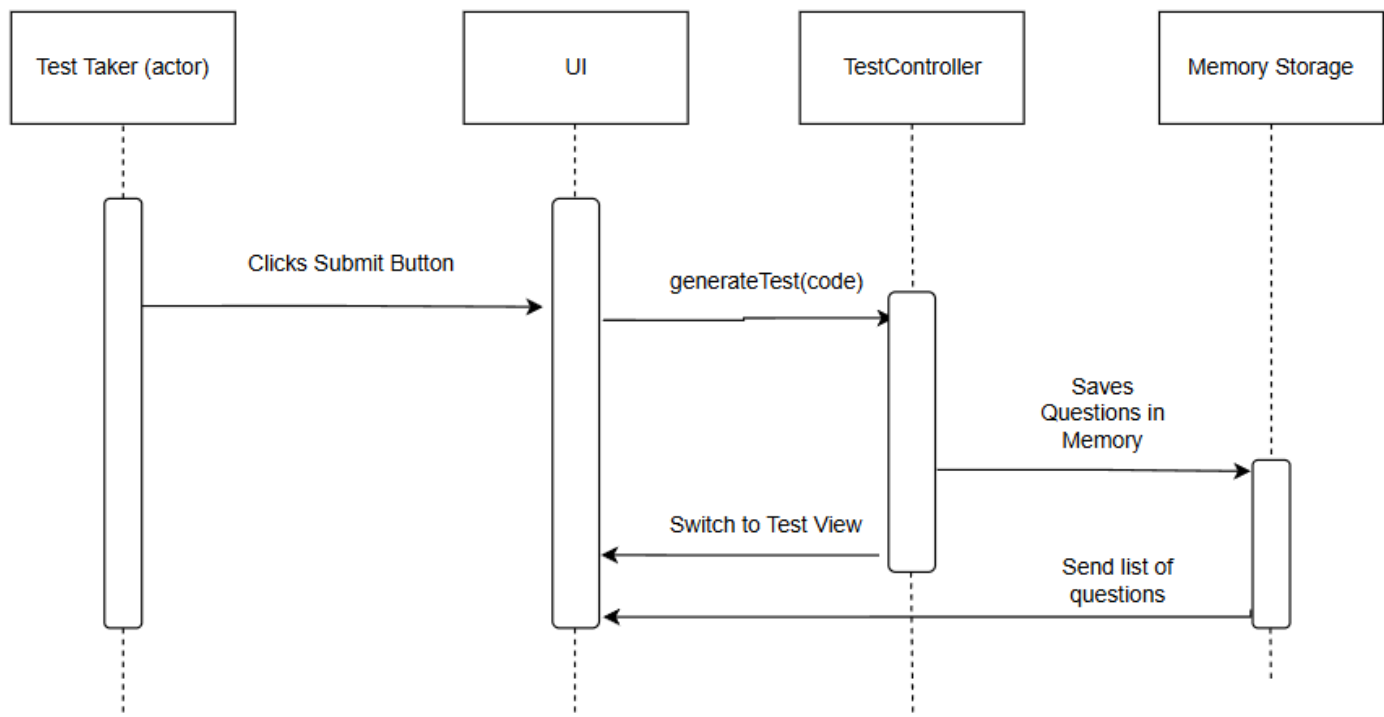
3.5 Design Constraints

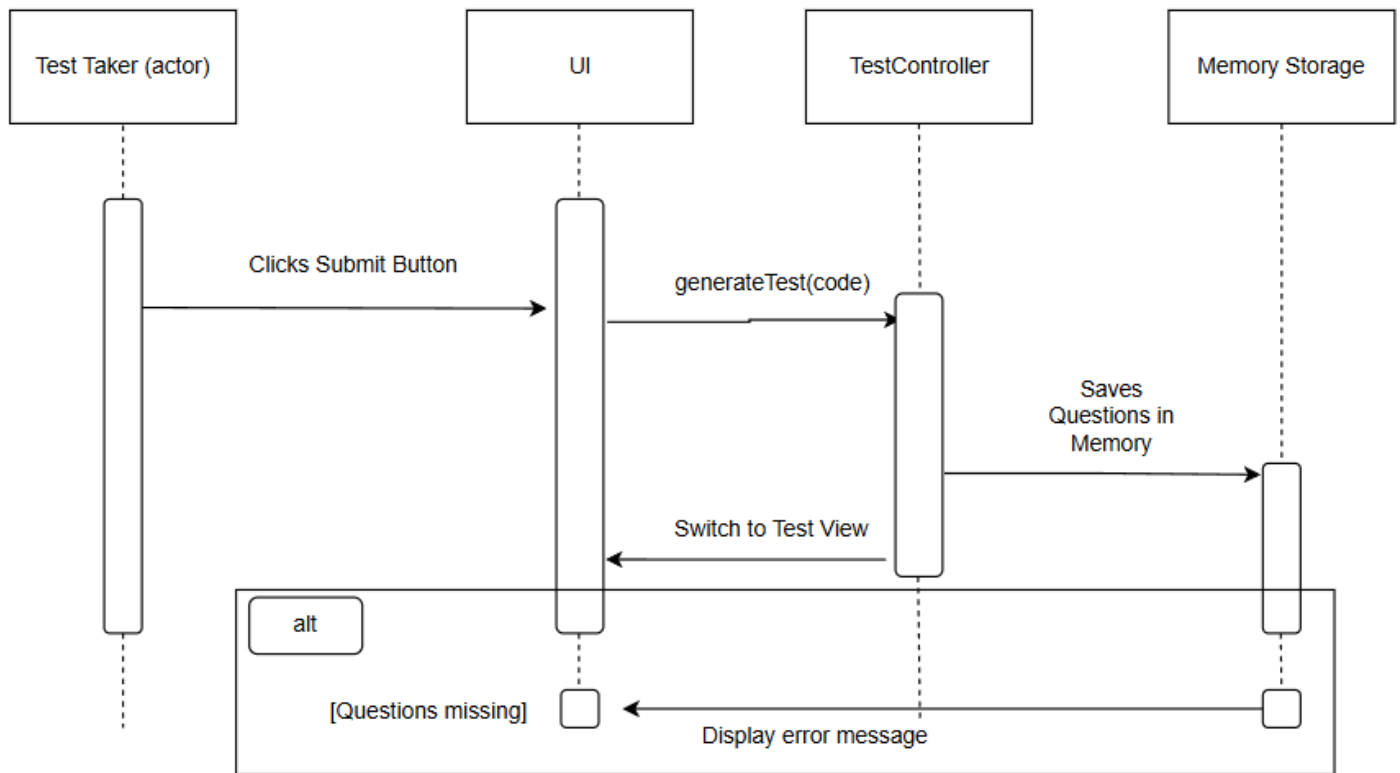
The software will be useable on macOS and Windows operating systems.

4. Analysis Models

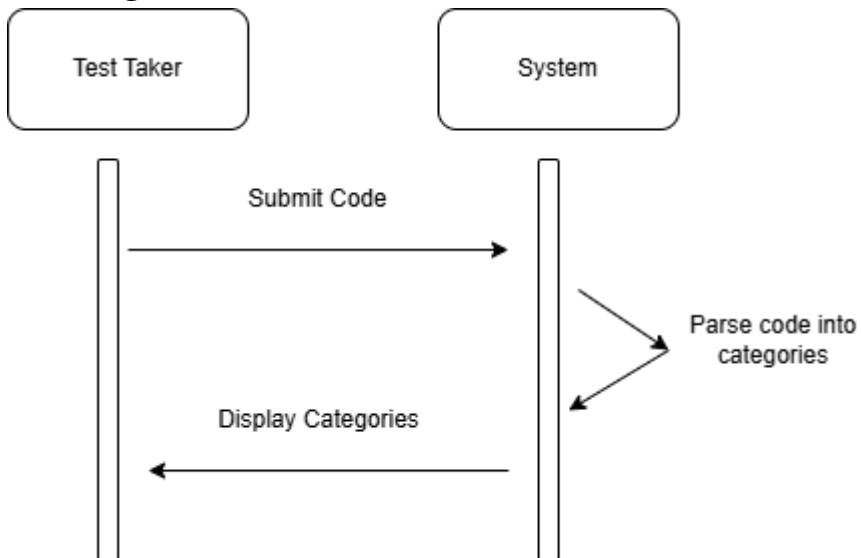
4.1 Sequence Diagrams

4.1.1 Diagram for Use Case #1

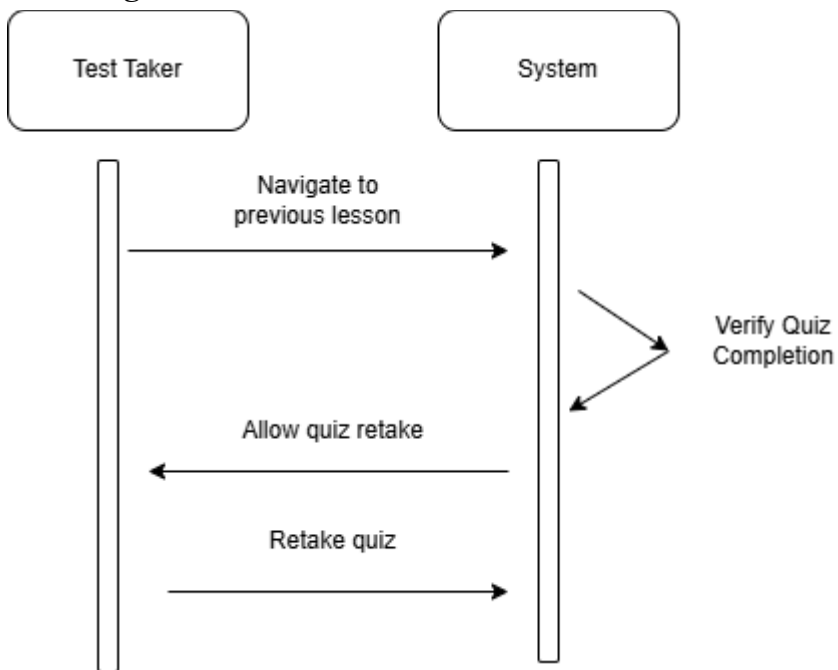




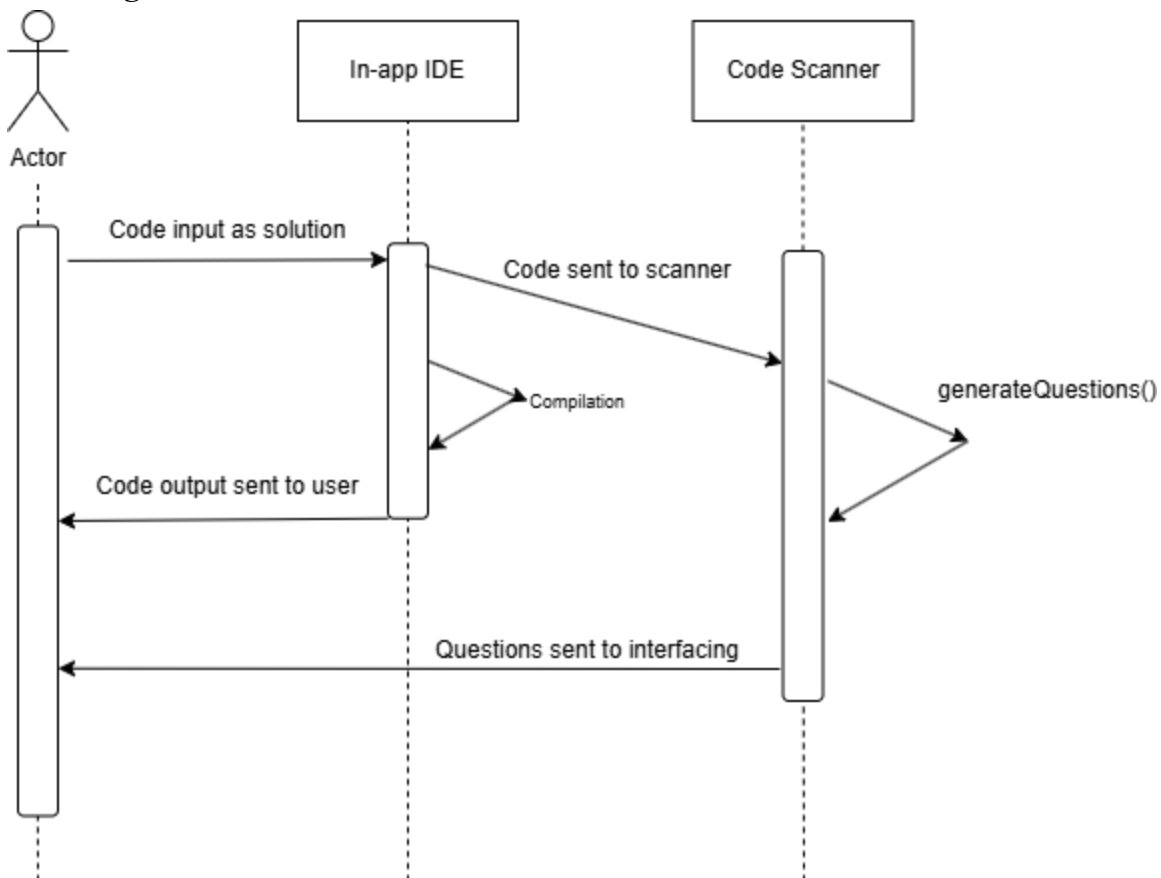
4.1.2 Diagram for Use Case #2



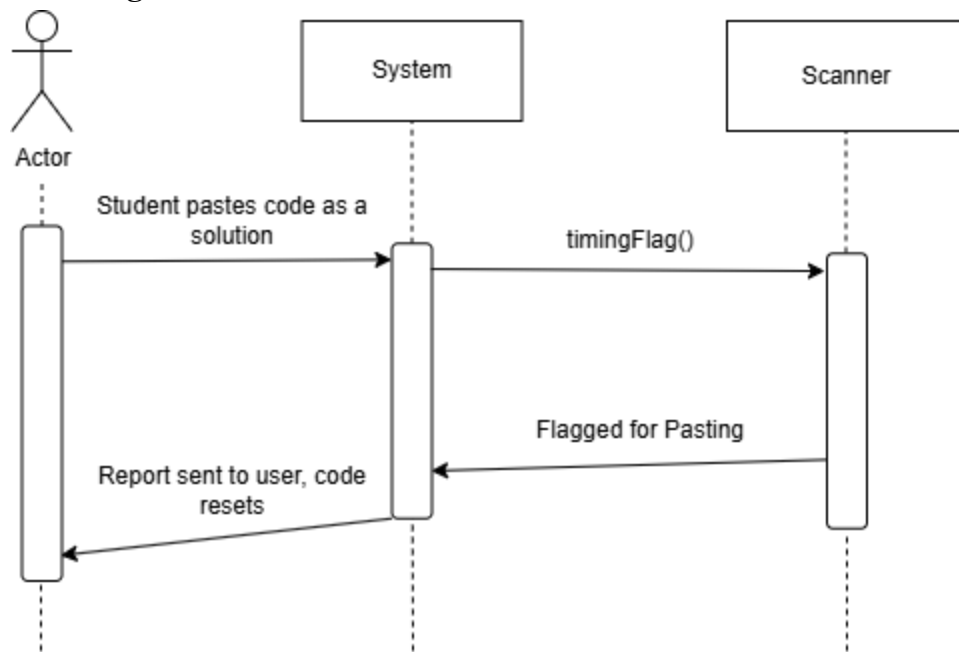
4.1.3 Diagram for Use Case #3



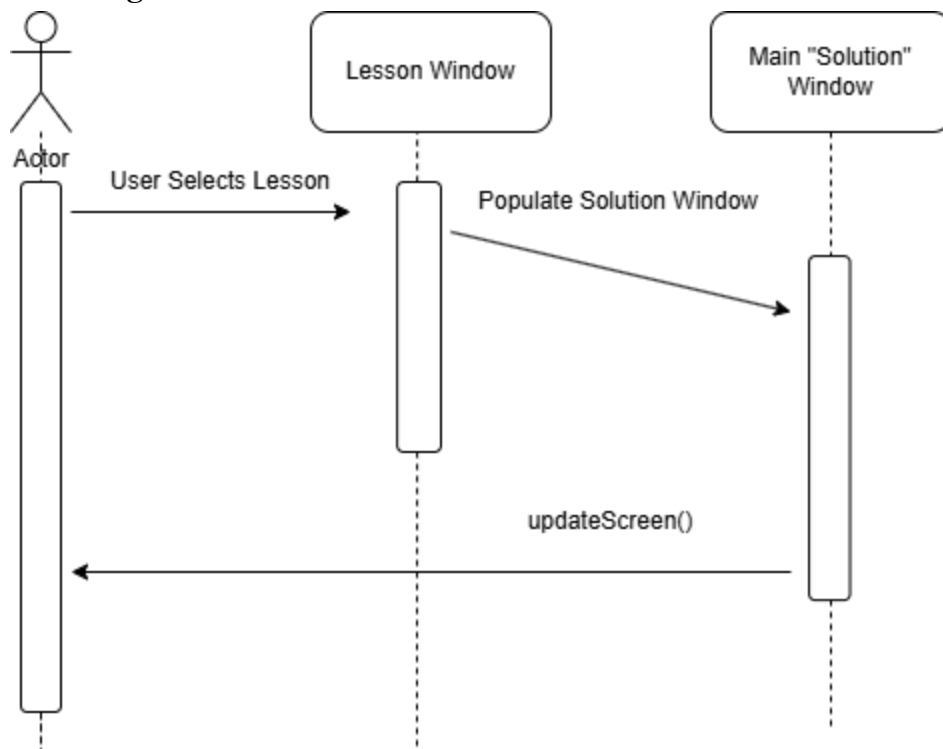
4.1.4 Diagram for Use Case #4



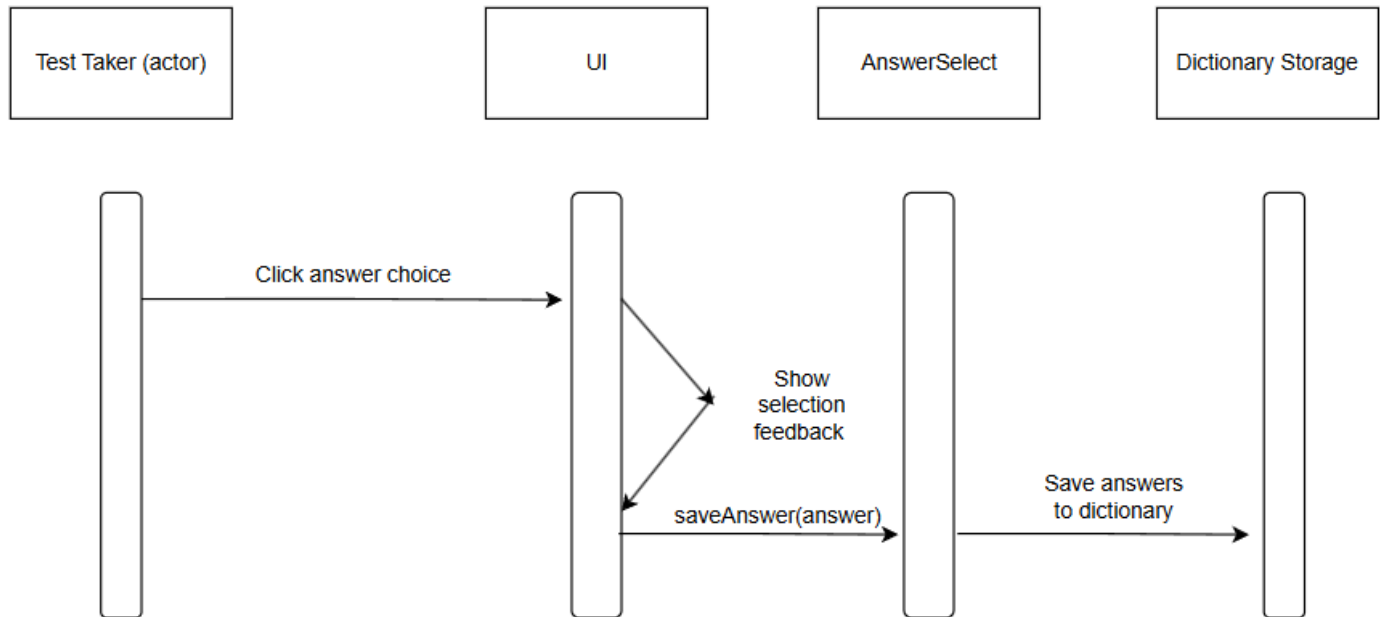
4.1.5 Diagram for Use Case #5



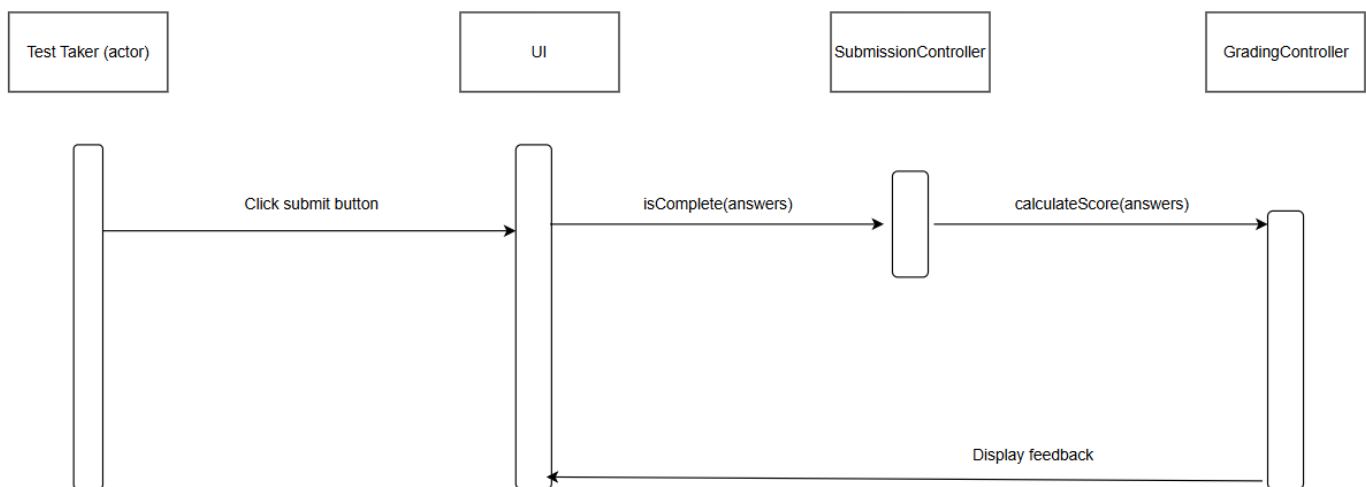
4.1.6 Diagram for Use Case #6

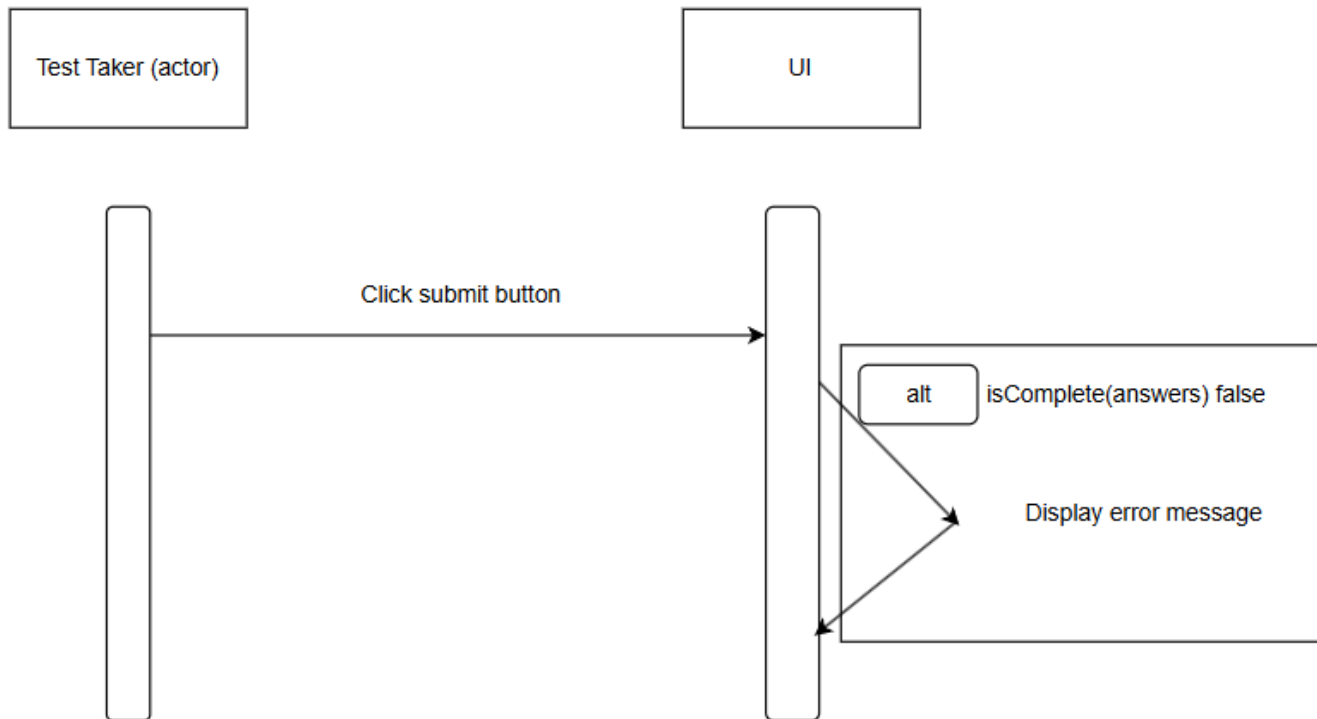


4.1.7 Diagram for Use Case #7



4.1.8 Diagram for Use Case #8





5. Change Management Process

Any change after the publishing of the 1.0 Version of the SRS is to be discussed by all team members before it is made. Any team member may make the change so long as it has been discussed in a meeting involving all members. These changes will be documented in the historical revision found at the top of page ii.

References

A. Appendices

A.1 Appendix 1

A.2 Appendix 2 – Meeting Notes